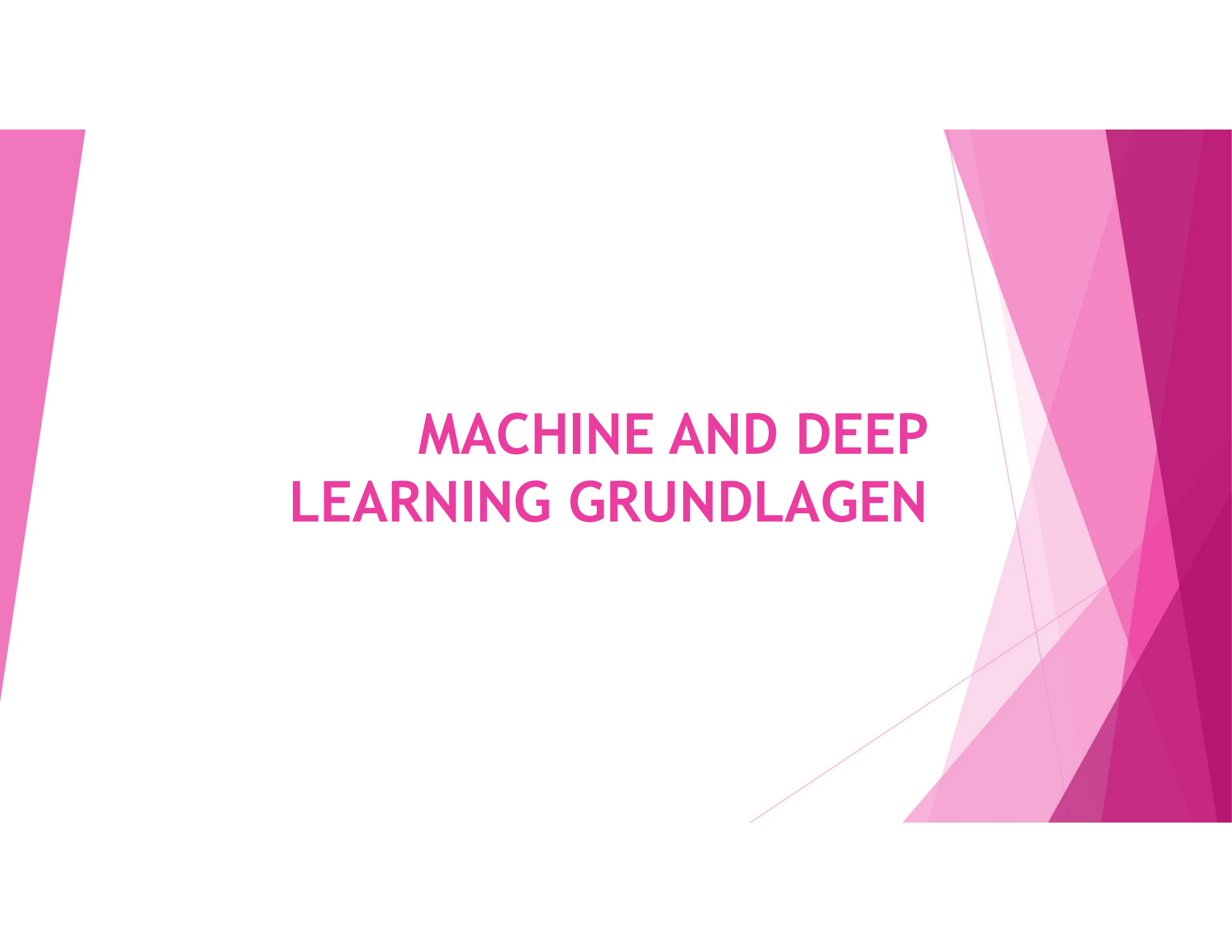


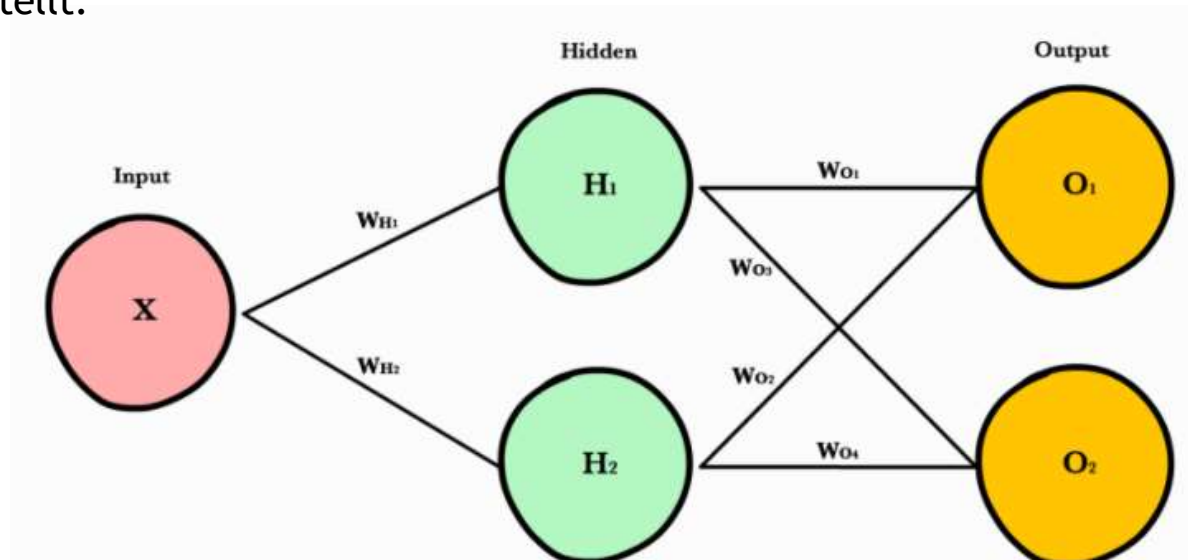


MACHINE AND DEEP LEARNING GRUNDLAGEN

NEURONALE NETZE -NEURAL NETWORKS

Ein neuronales Netzwerk ist eine Klasse von Algorithmen für maschinelles

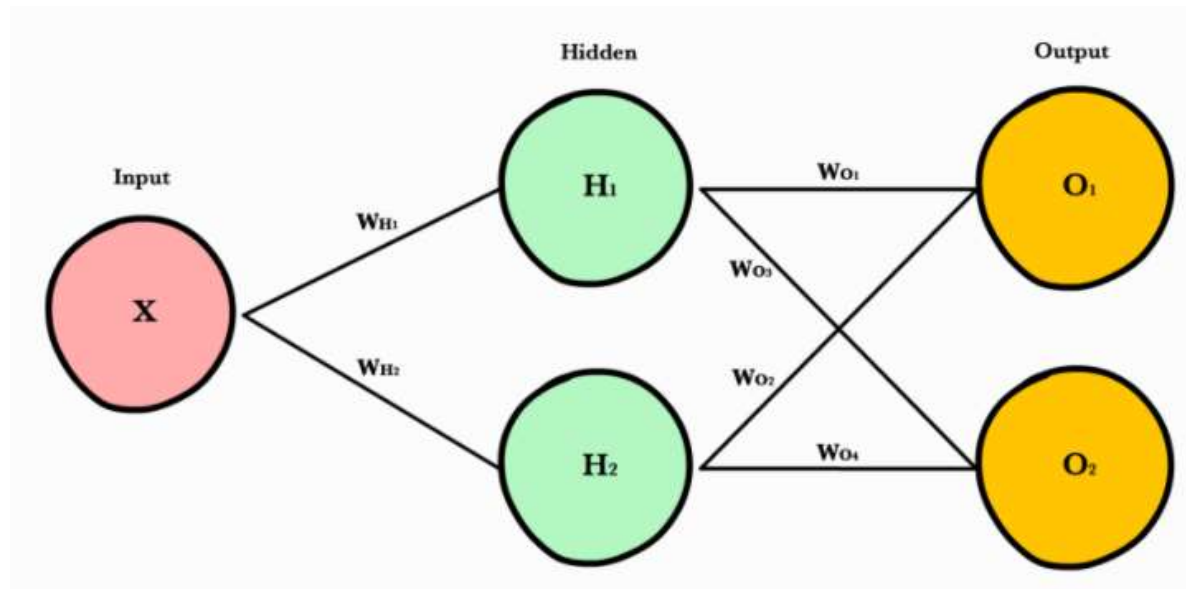
1. Lernen NN nimmt eine Eingabe entgegen,
2. leitet es durch mehrere Schichten versteckter Neuronen (Minifunktionen mit eindeutigen Koeffizienten, die gelernt werden müssen),
3. und gibt eine Vorhersage aus, die die kombinierte Eingabe aller Neuronen darstellt.



NEURONALE NETZE -NEURAL NETWORKS

A neural network is a class of machine learning algorithm

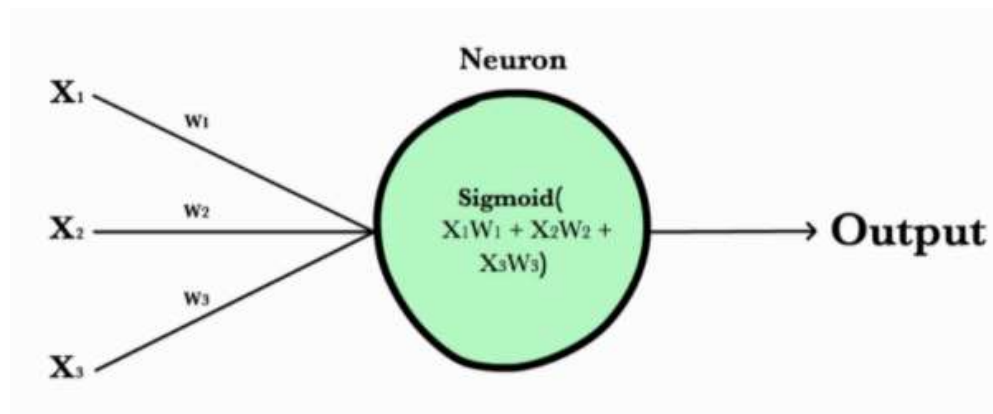
1. NN takes an input,
2. passes it through multiple layers of hidden neurons (mini-functions with unique coefficients that must be learned)
3. and outputs a prediction representing the combined input of all the neurons.



NEURON

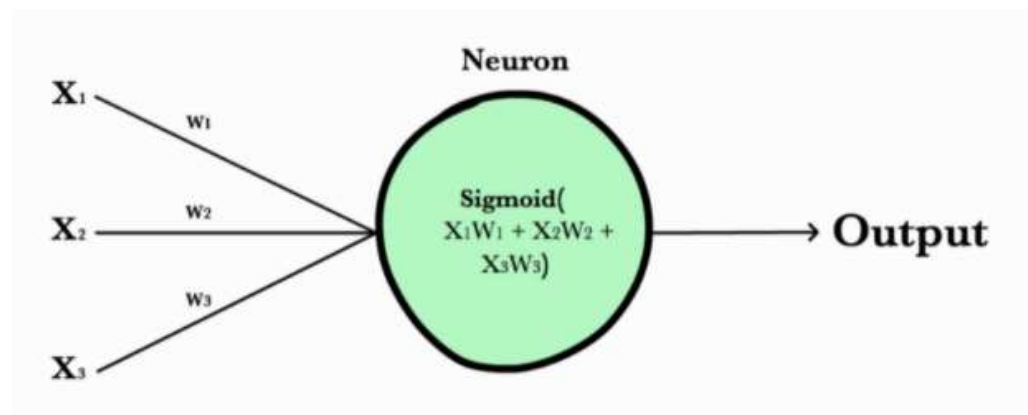
► Ein Neuron

1. nimmt eine Gruppe von gewichteten Eingaben,
2. fasst sie zusammen. Wendet eine Aktivierungsfunktion an und
3. gibt eine Ausgabe an die nächste Ebene zurück



NEURON

- ▶ A neuron
 1. takes a group of weighted inputs,
 2. Sum them
 3. applies an activation function, and returns an output to next layer
- ▶



SYNAPSE

- ▶ Synapsen sind wie Straßen, die Eingaben mit Neuronen, Neuronen mit Neuronen und Neuronen mit Ausgaben verbinden.
- ▶ Jede Verbindung zwischen zwei Neuronen hat eine eindeutige Synapse mit einem eindeutigen Gewicht

SYNAPSE

- ▶ Synapses are like roads connecting inputs to neurons, neurons to neurons, and neurons to outputs.
- ▶ Each connection between two neurons has a unique synapse with a unique weight attached to it

Gewichte

- ▶ Gewichte sind Werte, die die Stärke der Verbindung zwischen zwei Neuronen steuern
- ▶ Wenn die Eingaben zwischen Neuronen übertragen werden, werden die Gewichte zusammen mit einem zusätzlichen Wert (der Schwellenwert) auf die Eingaben angewendet.

Weights

- ▶ Weights are values that control the strength of the connection between two neurons
- ▶ when the inputs are transmitted between neurons, the weights are applied to the inputs along with an additional value (the bias)

Bias =Schwellenwert

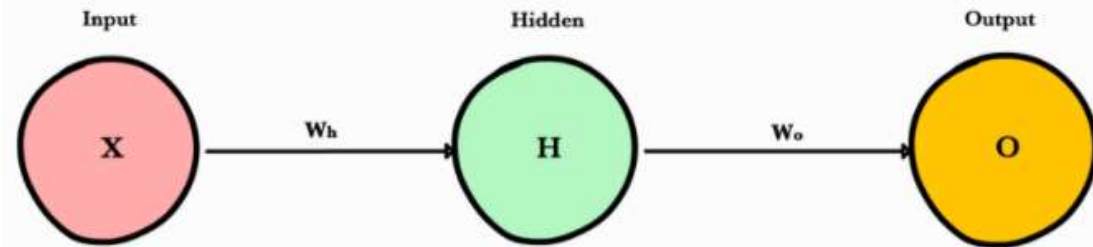
- ▶ Bias-Terme sind zusätzliche Konstanten, die an Neuronen angehängt und der gewichteten Eingabe hinzugefügt werden, bevor die Aktivierungsfunktion angewendet wird.
- ▶ Sie müssen auch von Ihrem Modell gelernt werden.

Bias

- ▶ Bias terms are additional constants attached to neurons and added to the weighted input before the activation function is applied.
- ▶ They must also be learned by your model.

Layers

Layers



Layers

Input Layer

Jedes Neuron in der Eingabeebene repräsentiert ein eindeutiges Attribut in Ihrem Datensatz

Hidden Layer

In der Regel vollständig verbunden und zwischen den Eingangs- und Ausgangsebenen und wendet eine Aktivierungsfunktion an, bevor die Ergebnisse weitergegeben werden.

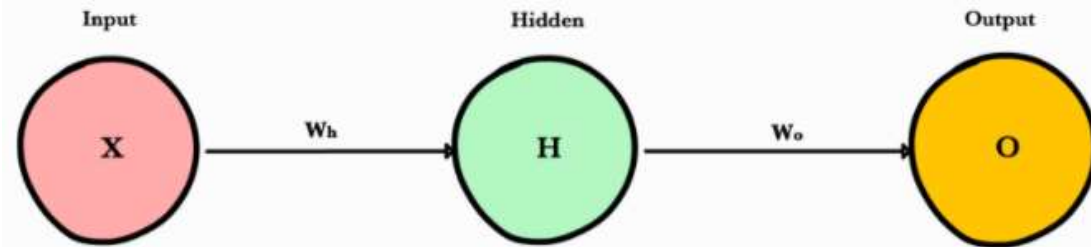
Output Layer

Die letzte Ebene gibt die Vorhersage des Modells zurück

Layers

Layers

Layers



Input Layer

Each neuron in the input layer represents a unique attribute in your dataset

Hidden Layer

typically fully-connected and its between the input and output layers and applies an activation function before passing on the results.

Output Layer

The final layer, returns the model's prediction.

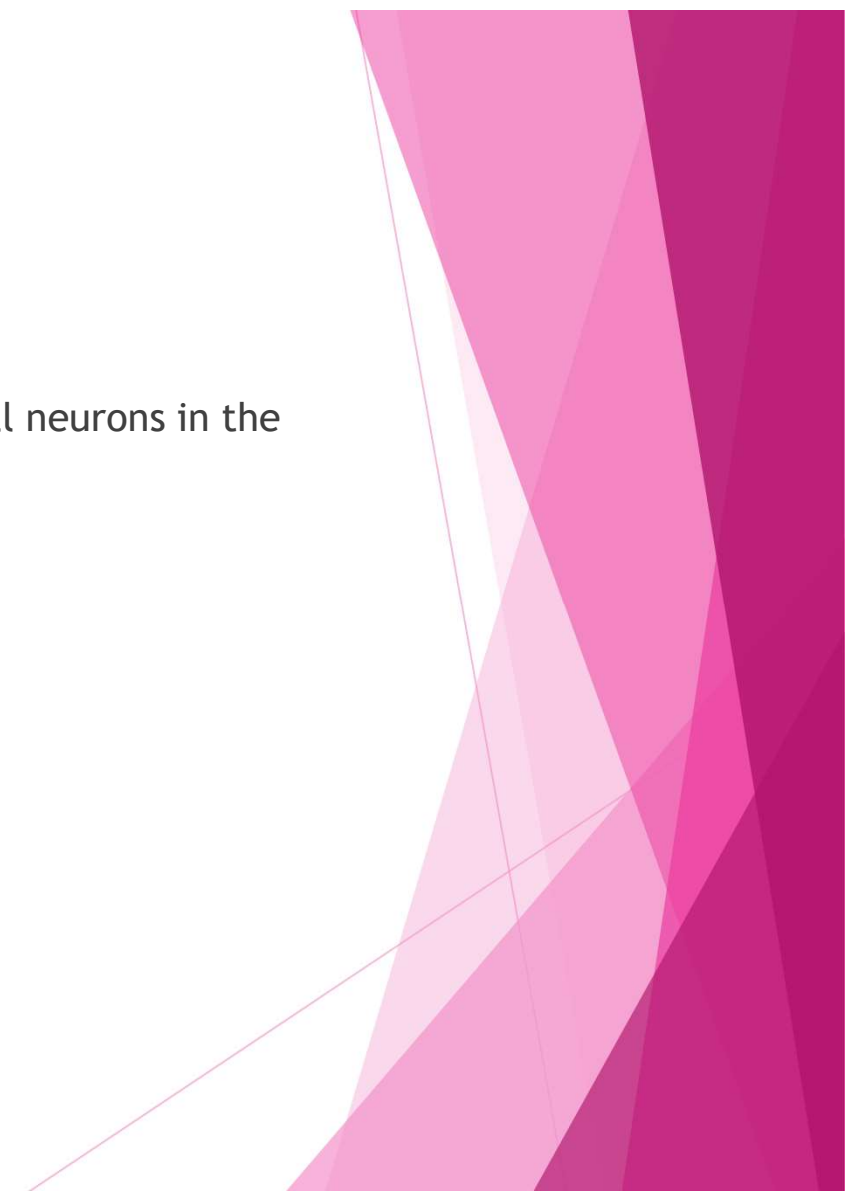
Gewichtete Eingabe

- ▶ Die Eingabe eines Neurons entspricht der Summe der gewichteten Ausgaben aller Neuronen in der vorherigen Schicht.



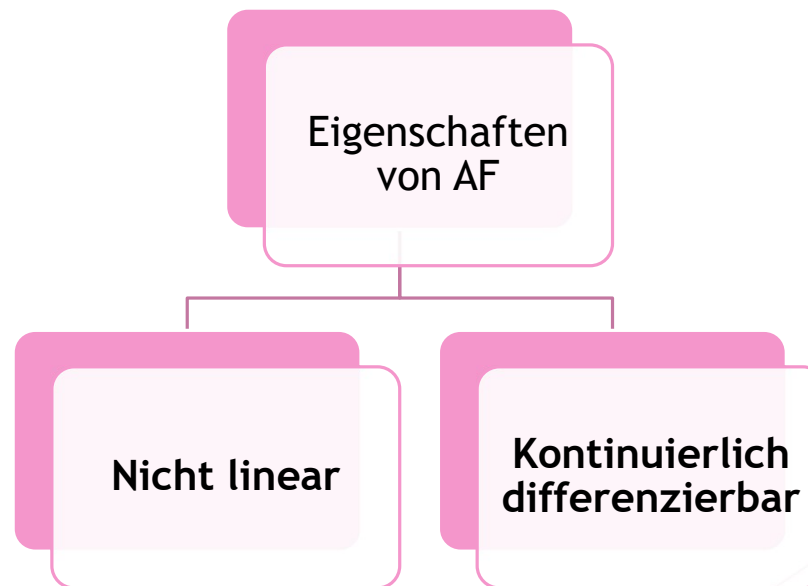
Weighted Input

- ▶ A neuron's input equals the sum of weighted outputs from all neurons in the previous layer.



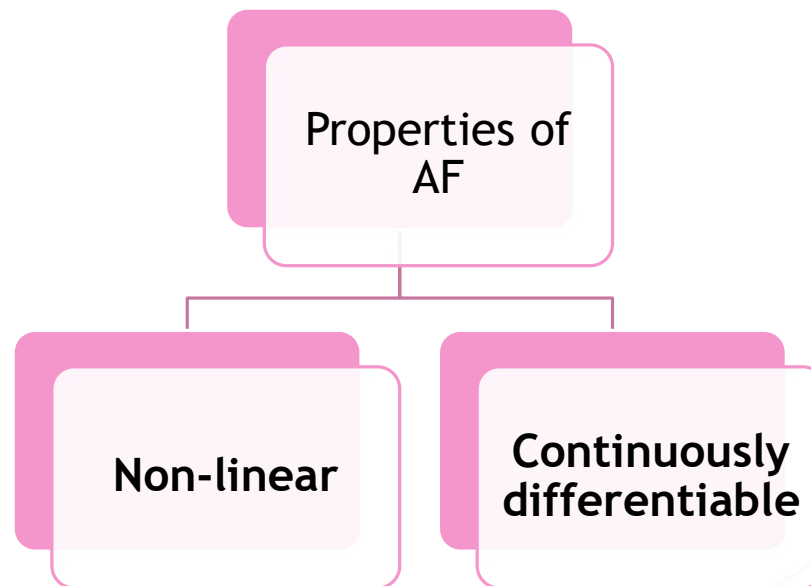
Activation Functions

- ▶ Aktivierungsfunktionen befinden sich in neuronalen Netzwerkschichten und ändern die empfangenen Daten, bevor sie an die nächste Schicht übergeben werden. Beliebte Aktivierungsfunktionen sind **Relu** und **Sigmoid**



Activation Functions

- ▶ Activation functions live inside neural network layers and modify the data they receive before passing it to the next layer.
- ▶ Popular activation functions include relu and sigmoid.



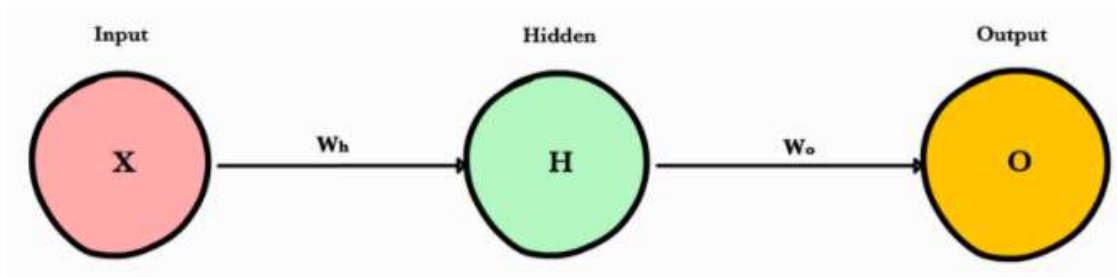
Verlustfunktionen - Loss Functions

- ▶ Eine Verlustfunktion oder Kostenfunktion ist ein Wrapper um die Vorhersagefunktion unseres Modells, der uns sagt, wie gut das Modell Vorhersagen für einen bestimmten Parametersatz treffen kann.
- ▶ Wir verwenden die Kostenfunktion, um unsere Parameter zu aktualisieren.
- ▶ Beliebte Verlustfunktionen sind: **MSE (L2)** und **Cross-Entropy Loss**.

Loss Functions

- ▶ A loss function, or cost function, is a wrapper around our model's predict function that tells us “how good” the model is at making predictions for a given set of parameters.
- ▶ We use the cost function to update our parameters.
- ▶ Popular loss functions include: MSE (L2) and Cross-entropy Loss.

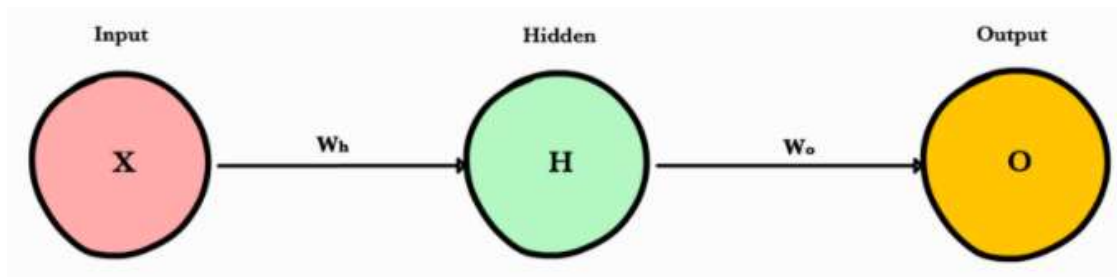
Einfaches Netzwerk



Schritte:

1. Berechnen Sie X multipliziert mit W_h
2. Wenden Sie die Aktivierungsfunktion an und übergeben Sie das Ergebnis H an die letzte Ebene
3. Berechnen Sie H multipliziert mit W_o

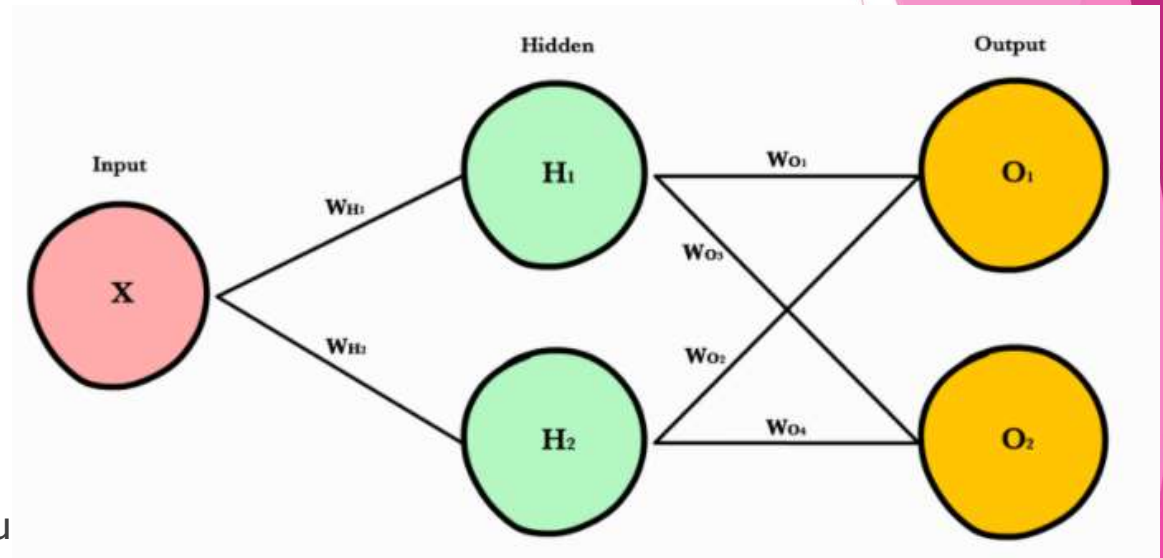
Simple Network



Steps

1. Calculate X multiplied by W_h
2. Apply the activation function and pass the result H to the final layer
3. Calculate H multiplied by W_o

Größeres Netzwerk



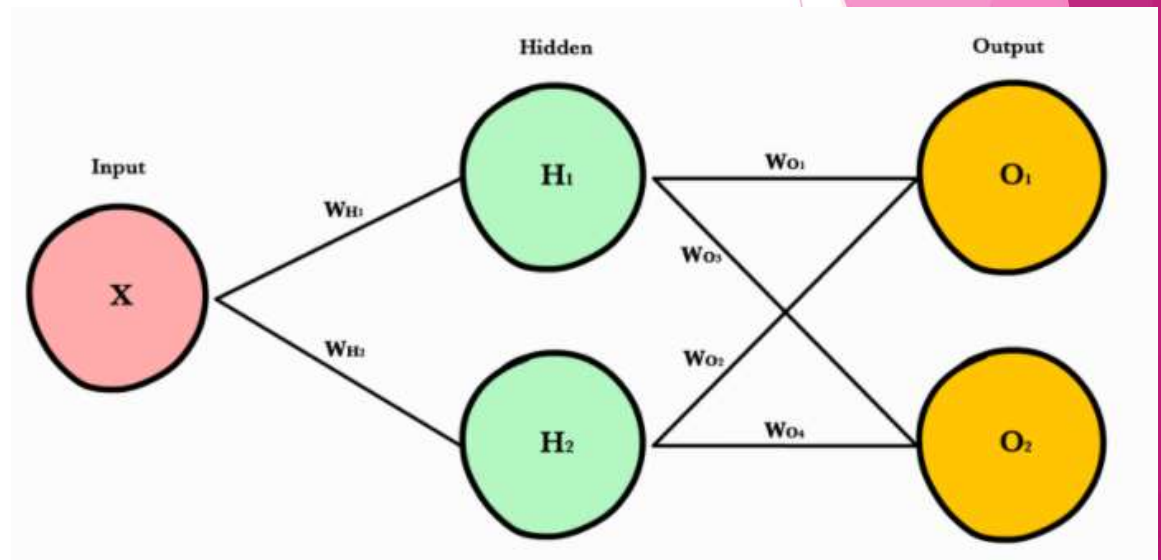
. Moderne neuronale Netze haben:

- viele versteckte Schichten, viele Neu
- viele Variablen pro Eingabe,
- viele Eingaben pro Trainingssatz,
- und viele Ausgabevariablen, die vorhergesagt werden müssen t

Larger Network

Modern neural networks have :

- ▶ many hidden layers,
- ▶ many neurons per layer,
- ▶ many variables per input,
- ▶ many inputs per training set,
- ▶ and many output variables to predict.



Schwellenwert - Bias Terms

Mit **Bias-Begriffen**

können wir die Aktivierungsausgaben unseres Neurons nach links und rechts verschieben.

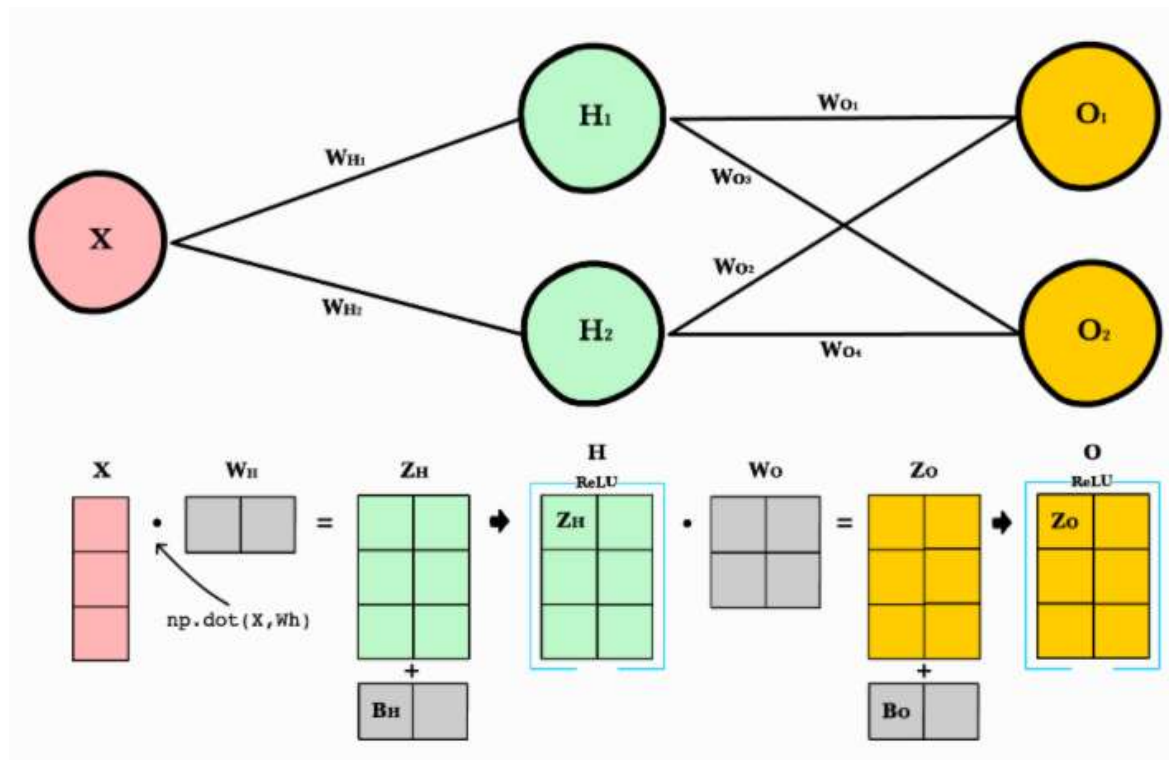


Bias Terms

- ▶ Bias terms allow us to shift our neuron's activation outputs left and right. This helps us model datasets that do not necessarily pass through the origin.

Working with Matrices

- ▶ Matrices are used to store the inputs, weights, and biases



**NICHT
Prüfungsrelevant**

Matrix dimensions

Var	Name	Dimensions	Explanation
X	Input	(3, 1)	Includes 3 rows of training data, and each row has 1 attribute (height, price, etc.)
W_h	Hidden weights	(1, 2)	These dimensions are based on number of rows equals the number of attributes for the observations in our training set. The number columns equals the number of neurons in the hidden layer. The dimensions of the weights matrix between two layers is determined by the sizes of the two layers it connects. There is one weight for every input-to-neuron connection between the layers.
B_h	Hidden bias	(1, 2)	Each neuron in the hidden layer has its own bias constant. This bias matrix is added to the weighted input matrix before the hidden layer applies ReLU.
Z_h	Hidden weighted input	(1, 2)	Computed by taking the dot product of X and W_h . The dimensions (1,2) are required by the rules of matrix multiplication. Z_h takes the rows of in the inputs matrix and the columns of weights matrix. We then add the hidden layer bias matrix B_h .
H	Hidden activations	(3, 2)	Computed by applying the Relu function to Z_h . The dimensions are (3,2) – the number of rows matches the number of training samples and the number of columns equals the number of neurons. Each column holds all the activations for a specific neuron.
W_o	Output weights	(2, 2)	The number of rows matches the number of hidden layer neurons and the number of columns equals the number of output layer neurons. There is one weight for every hidden-neuron-to-output-neuron connection between the layers.
B_o	Output bias	(1, 2)	There is one column for every neuron in the output layer.
Z_o	Output weighted input	(3, 2)	Computed by taking the dot product of H and W_o and then adding the output layer bias B_o . The dimensions are (3,2) representing the rows of in the hidden layer matrix and the columns of output layer weights matrix.
O	Output activations	(3, 2)	Each row represents a prediction for a single observation in our training set. Each column is a unique attribute we want to predict. Examples of two-column output predictions could be a company's sales and units sold, or a person's height and weight.

NICHT
Prüfungsrelevant

Ziele der Backpropagation

- ▶ Passen Sie jedes Gewicht im Netzwerk proportional dazu an, wie viel es zum Gesamtfehler beiträgt.
- ▶ Wenn wir den Fehler jedes Gewichts iterativ reduzieren, haben wir schließlich eine Reihe von Gewichten, die gute Vorhersagen liefern.

Goals of the Backpropagation

- ▶ adjust each weight in the network in proportion to how much it contributes to overall error.
- ▶ If we iteratively reduce each weight's error, eventually we'll have a series of weights that produce good predictions.

Chain rule refresher

**NICHT
Prüfungsrelevant**

- ▶ As seen above, forward propagation can be viewed as a long series of nested equations.
- ▶ If you think of feed forward this way, then backpropagation is merely an application of Chain rule to find the Derivatives of cost with respect to any variable in the nested equation

Chain Rule

**NICHT
Prüfungsrelevant**

Function	Formula	Derivative
Weighted input	$Z = XW$	$Z'(X) = W$ $Z'(W) = X$
ReLU activation	$R = \max(0, Z)$	$R'(Z) = \begin{cases} 0 & Z < 0 \\ 1 & Z > 0 \end{cases}$
Cost function	$C = \frac{1}{2}(\hat{y} - y)^2$	$C'(\hat{y}) = (\hat{y} - y)$

Examples of Chain rule

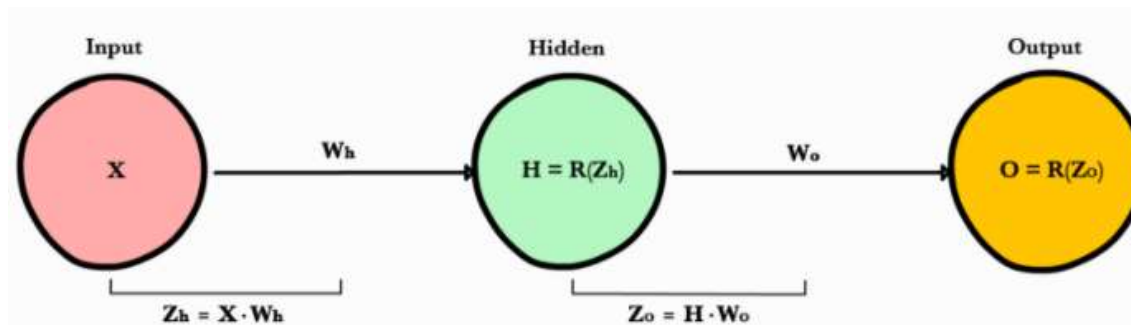
**NICHT
Prüfungsrelevant**

Given a forward propagation function:

$$f(x) = A(B(C(x)))$$

A, B, and C are activation functions at different layers. Using the chain rule we easily calculate the derivative of $f(x)$ with respect to x :

$$f'(x) = f'(A) \cdot A'(B) \cdot B'(C) \cdot C'(x)$$



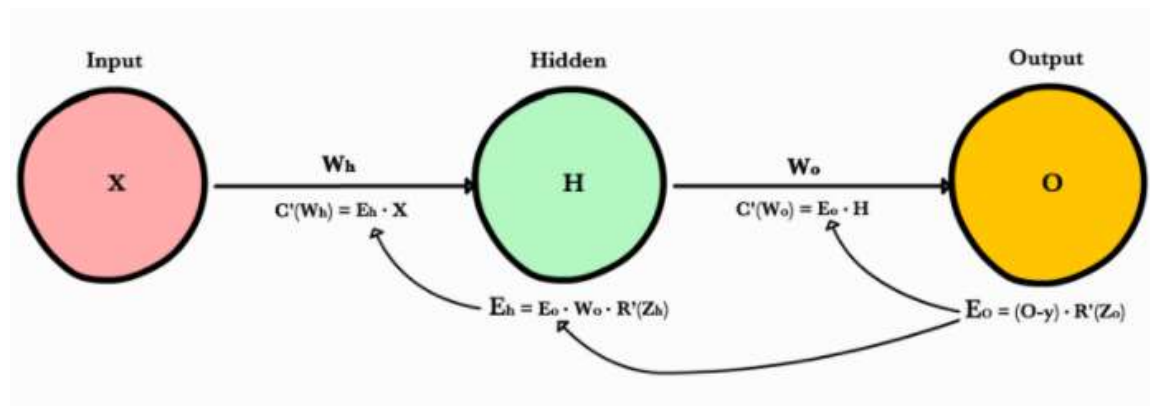
What is the derivative of cost with respect to W_o ?

$$\begin{aligned} C'(W_o) &= C'(\hat{y}) \cdot \hat{y}'(Z_o) \cdot Z_o'(W_o) \\ &= (\hat{y} - y) \cdot R'(Z_o) \cdot H \end{aligned}$$

**NICHT
Prüfungsrelevant**

3 equations that together form the foundation of backpropagation

Output Layer Error	$E_o = (O - y) \cdot R'(Z_o)$
Hidden Layer Error	$E_h = E_o \cdot W_o \cdot R'(Z_h)$
Cost-Weights Deriv	$LayerError \cdot LayerInput$



ReLU Hidden Layer-Aktivierungsfunktion

- ▶ Die gleichgerichtete lineare Aktivierungsfunktion oder **ReLU**-Aktivierungsfunktion ist möglicherweise die häufigste Funktion, die für verborgene Schichten verwendet wird.
- ▶ Es ist üblich, weil es ist:
 - ▶ einfach zu implementieren und
 - ▶ wirksam bei der Überwindung der Einschränkungen anderer bisher beliebter Aktivierungsfunktionen wie Sigmoid und Tanhweniger.
 - ▶ ReLu ist weniger anfällig für verschwindende Gradienten Farbverläufe (**vanishing gradients**), die verhindern, dass tiefe Modelle trainiert werden.
 - ▶ ABER es kann unter anderen Problemen wie gesättigten oder "toten" Einheiten („dead“ units) leiden.

ReLU Hidden Layer Activation Function

- ▶ The rectified linear activation function, or ReLU activation function, is perhaps the most common function used for hidden layers.
- ▶ It is common because it is :
 1. simple to implement and
 2. effective at overcoming the limitations of other previously popular activation functions, such as Sigmoid and Tanh
 3. less susceptible to vanishing gradients that prevent deep models from being trained.

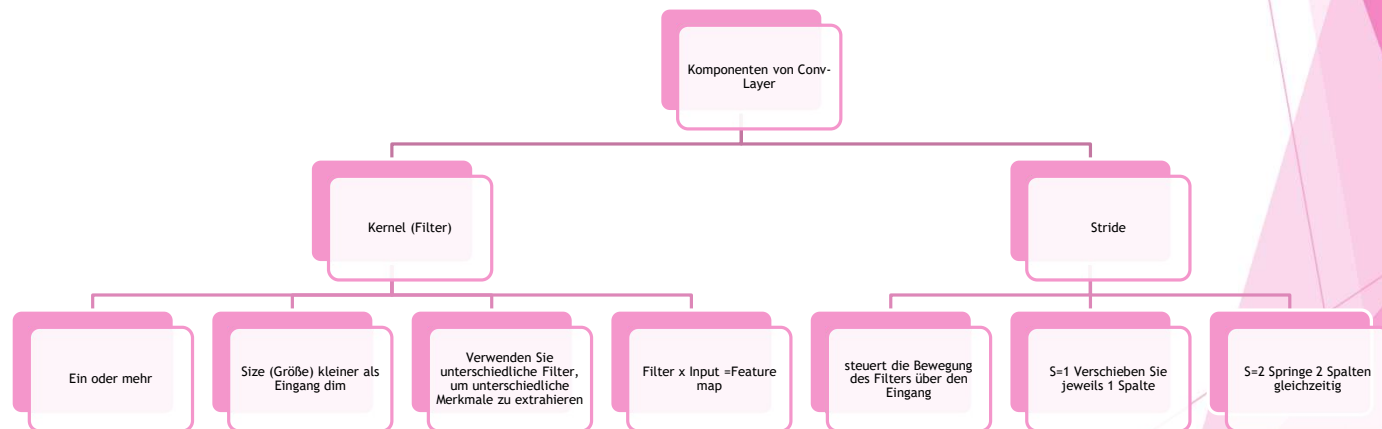
BUT it can suffer from other problems like saturated or “*dead*” units.

Layers

- BatchNorm
- Convolution
- Dropout
- Linear
- LSTM
- Pooling
- RNN

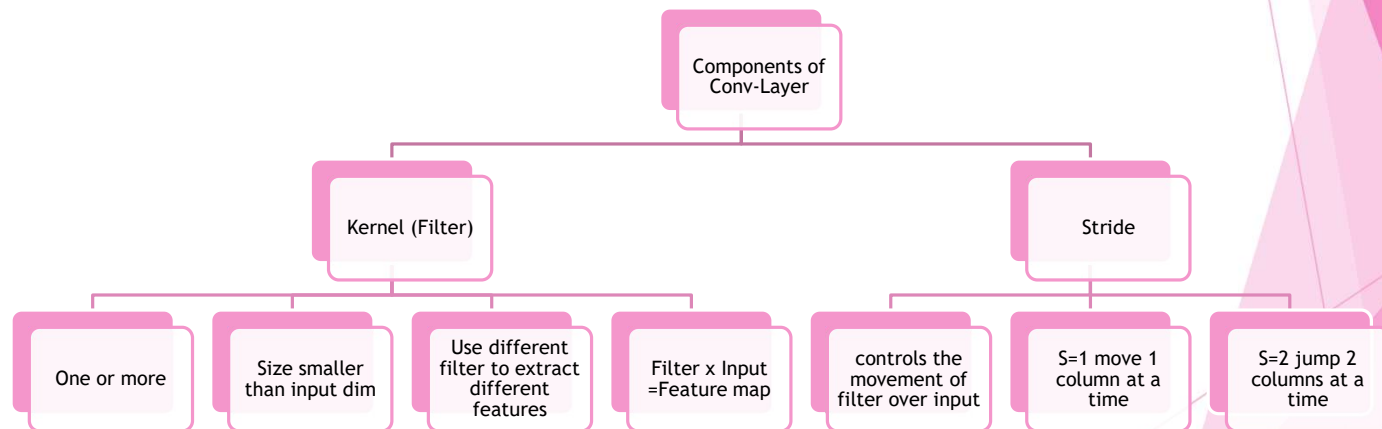
Faltungsschichten CNN

- Eine Faltung ist eine lineare Operation, bei der das Gewicht (Kernel / Filter) mit der Eingabe multipliziert wird und die den größten Teil der Schwerlastarbeit erledigt.



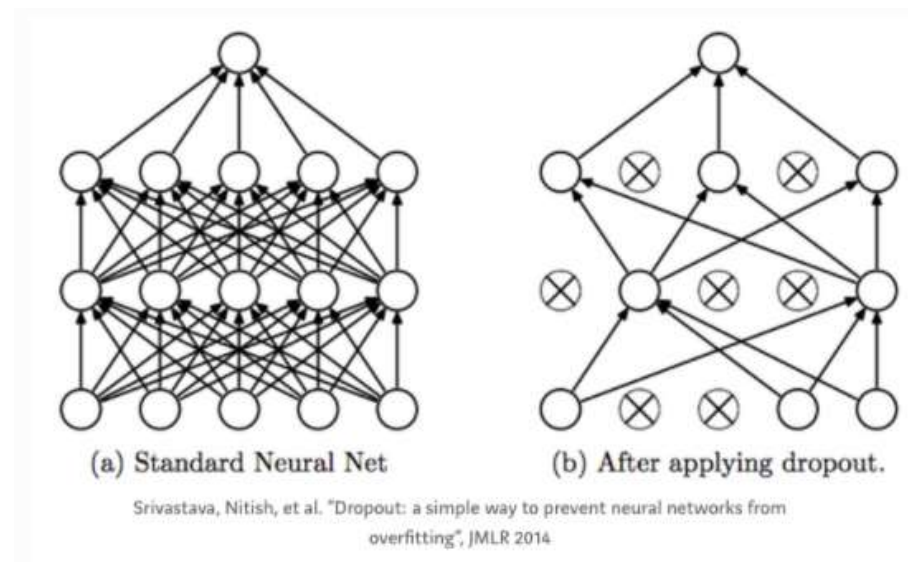
Convolutional layers CNN

- a convolution is a linear operation that involves multiplication of weight (kernel/filter) with the input and it does most of the heavy lifting job.



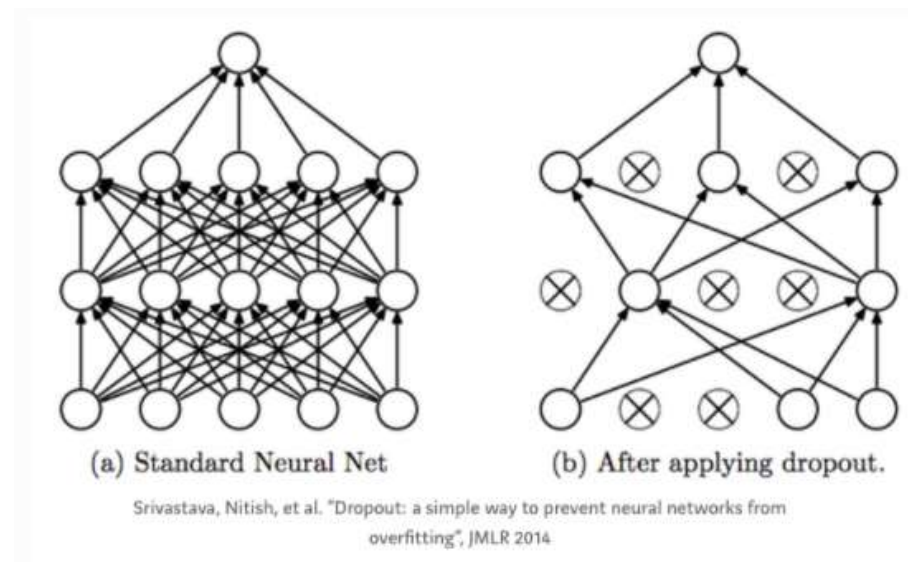
Dropout layer

- ▶ Eine Dropout-Ebene nimmt die Ausgabe der Aktivierungen der vorherigen Ebene und setzt zufällig einen bestimmten Bruchteil (Dropout-Rate) der Aktivierungen auf 0, wodurch sie abgebrochen oder "gelöscht" werden.
- ▶ Es ist eine übliche Regularisierungstechnik, die verwendet wird, um eine Überanpassung in neuronalen Netzen zu verhindern.



Dropout layer

- ▶ A dropout layer takes the output of the previous layer's activations and randomly sets a certain fraction (dropout rate) of the activations to 0, cancelling or 'dropping' them out.
- ▶ It is a common regularization technique used to prevent overfitting in Neural Networks.



Erklärung Dropout layer

The dropout rate is the tunable hyperparameter that is adjusted to measure performance with different values. It is typically set between 0.2 and 0.5 (but may be arbitrarily set).

Dropout is only used during training; At test time, no activations are dropped, but scaled down by a factor of dropout rate. This is to account for more units being active during test time than training time.

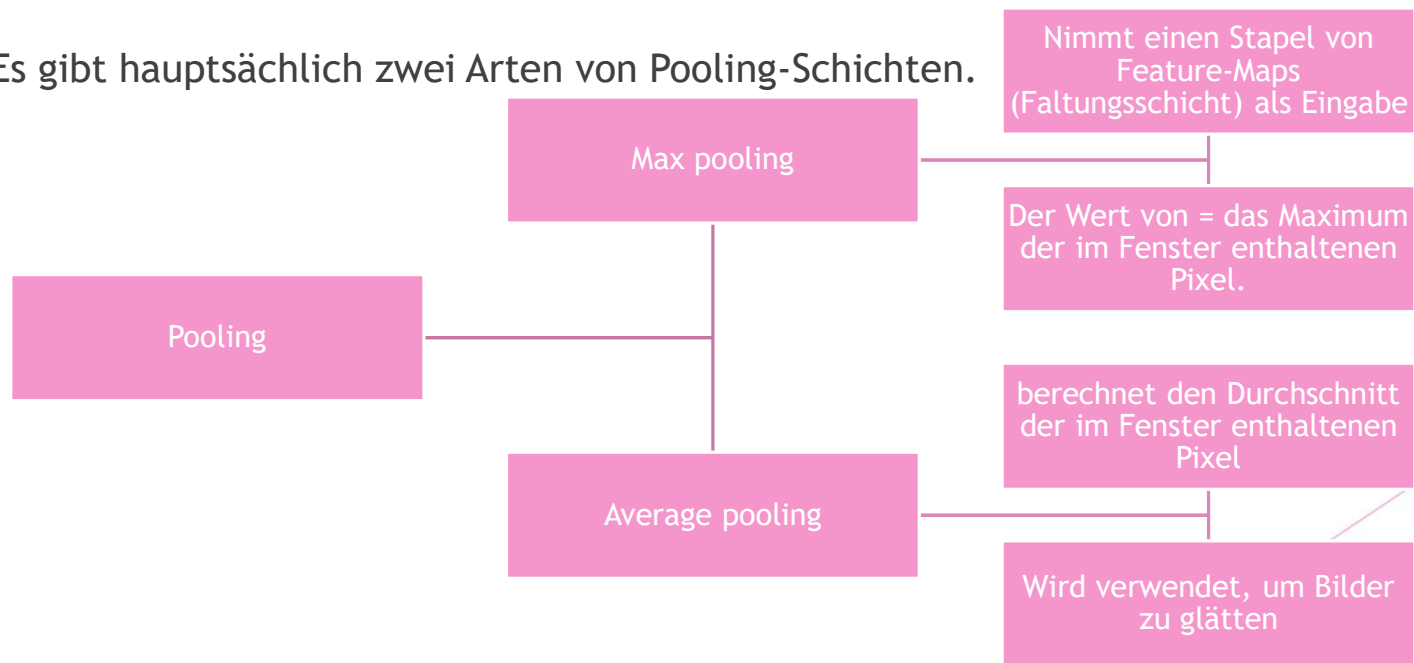
For example:

- A layer in a neural net outputs a tensor (matrix) A of shape $(\text{batch_size}, \text{num_features})$.
- The dropout rate of the layer is set to 0.5 (50%).
- A random 50% of the values in A will be set to 0.
- These will then be multiplied with the weight matrix to form the inputs to the next layer.

The premise behind dropout is to introduce noise into a layer in order to disrupt any interdependent learning or coincidental patterns that may occur between units in the layer, that aren't significant.

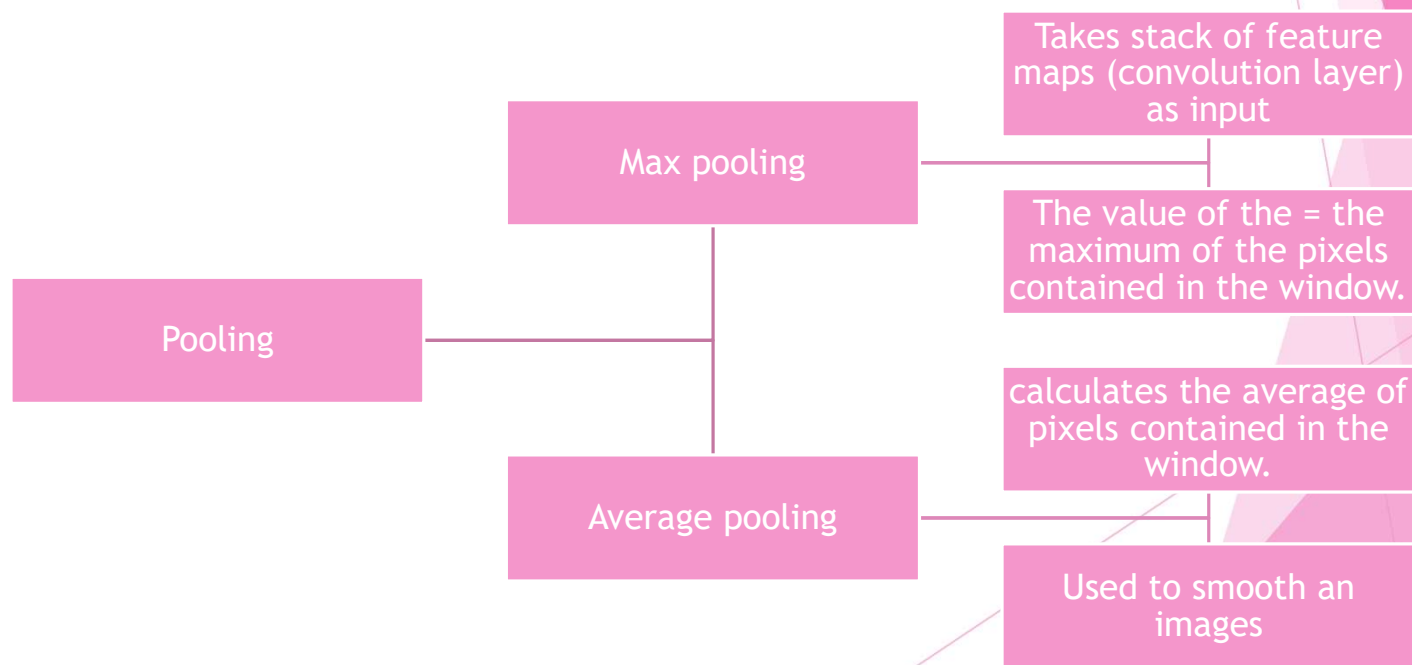
Pooling

- ▶ Pooling-Schichten sind Methoden, um diese hohe Dimension zu reduzieren
- ▶ Genau wie bei der Faltungsschicht gibt es Kernelgröße und Stride (Schrittweite) den meisten Fällen beträgt die Größe des Kernels 2×2 und der Schritt 2.
- ▶ Es gibt hauptsächlich zwei Arten von Pooling-Schichten.



Pooling

- ▶ Pooling layers are methods for reducing this high dimensionally
- ▶ Just like the convolution layer, there is kernel size and stride
- ▶ For most of the cases the size of the kernel will be 2×2 and the stride of 2. There are mainly two types of pooling layers.



Vielen Dank für Ihre Aufmerksamkeit
Thank you for your attention

